EVALUACIÓN

Taller de Lenguajes de Programación

Preguntas y respuestas

1. En un JSP, ¿qué diferencia hay entre estas dos líneas si 3 personas abren la página?

Respuesta: La variable a se declara como atributo del servlet y puede conservar su valor entre peticiones. La variable b es local a cada petición y vuelve a inicializarse en cero.

2. ¿Cuántas filas quedan en usuarios?

Respuesta: Quedan 2 filas. En MySQL, una columna con restricción UNIQUE permite varios valores NULL.

```
INSERT INTO usuarios (correo)
VALUES (NULL), (NULL);
```

4. Carlos tiene 500 puntos y 2 canjes. ¿Qué suma muestra la consulta?

Respuesta: Muestra 1000 puntos para Carlos, porque sus 500 puntos se repiten en las dos filas generadas por el JOIN.

```
SELECT a.nombre, SUM(a.puntos)
FROM estudiantes a
JOIN historial_canjes h
ON h.id_estudiante = a.id
GROUP BY a.nombre;
```

6. Se abre /juegos/5. ¿Qué devuelve request.getPathInfo()?

Respuesta: Devuelve /5, porque es la parte de la ruta que aparece después del patrón /juegos/*.

7. ¿Qué ocurre con este alias?

Respuesta: Se produce un error porque la columna tiene como nombre de consulta estudiante, no nombre. Debe usarse rs.getString("estudiante").

```
rs = conn.prepareStatement(
    "SELECT e.nombre AS estudiante FROM estudiantes e"
).executeQuery();
```

```
rs.next();
rs.getString("nombre");
```

8. Luis tenía 200 puntos. ¿Cuántos puntos tiene después de ejecutar el código?

Respuesta: Conserva sus 200 puntos, porque se desactiva el autocommit y no se ejecuta commit(). Al cerrarse la conexión sin confirmar la transacción, el cambio se revierte.

9. Para ahorrar código, un servlet que elimina registros llama a doPost() desde doGet(). ¿Qué riesgo introduce?

Respuesta: Permite eliminar registros mediante una petición GET, que podría activarse accidentalmente o sin autorización adecuada. Las operaciones que modifican datos deben protegerse y no ejecutarse mediante GET.

11. Carlos tiene 500 puntos. ¿Qué devuelve executeUpdate() con Connector/J?

Respuesta: Devuelve 1, porque se actualiza una fila: la correspondiente al estudiante con ID 1.

14. ¿Qué ocurre al recorrer un CachedRowSet después de cerrar la conexión?

Respuesta: Se pueden recorrer sus datos porque CachedRowSet almacena una copia en memoria y permite trabajar sin mantener abierta la conexión.

15. Se abre saludo.jsp sin enviar el parámetro nombre. ¿Qué muestra cada línea?

Respuesta: request.getParameter("nombre") puede mostrar null. En cambio, ${param.nombre} mediante JSP EL muestra una cadena vacía.

16. El título es "Assassin's Creed". ¿Qué se ve en el campo del formulario?

Respuesta: El apóstrofo puede cerrar incorrectamente el atributo HTML delimitado por comillas simples y provocar que el campo se muestre mal. Se deben escapar los caracteres especiales.

18. Se descuentan puntos y después falla el INSERT del canje. ¿Cómo queda la base de datos con autocommit predeterminado?

Respuesta: Los puntos quedan descontados, aunque el canje no se registre. Con autocommit activado, la actualización se confirma por separado y no se revierte automáticamente cuando falla el INSERT.

22. ¿Qué ocurre al ejecutar el INSERT si solo se asigna el nombre "Ana"?

Respuesta: Se produce un error porque no se asignó un valor al segundo parámetro (puntos) del PreparedStatement.

23. ¿Cuántos juegos se insertan en el lote?

Respuesta: Se agrega un solo juego al lote: X3, porque addBatch() está fuera del bucle y se ejecuta una sola vez.

24. Carlos tiene 500 puntos. ¿Qué devuelve la consulta?

Respuesta: Devuelve 500, correspondiente a los puntos del estudiante con ID 1.

26. Login hecho con concatenación. Si el usuario escribe exactamente ' OR '1'='1, ¿qué ocurre?

Respuesta: La condición SQL se altera y puede resultar verdadera para todas las filas. Se produce una vulnerabilidad de inyección SQL que permite evadir el filtro de inicio de sesión.

27. ¿Con qué código HTTP llega esta respuesta?

Respuesta: Con el código HTTP 200 OK, porque el servlet no establece otro código de estado y solo imprime un mensaje de error en el contenido.

28. La columna puntos es INT DEFAULT 0 y admite NULL. ¿Qué valor tiene cada fila?

Respuesta: Pepe: 0, porque no se especificaron los puntos y se aplica el valor predeterminado. Lola: NULL, porque se indicó explícitamente ese valor.

36. ¿Qué ocurre en la segunda línea?

Respuesta: Se produce una IllegalStateException, porque no se pueden utilizar simultáneamente getWriter() y getOutputStream() en la misma respuesta.

```
response.getWriter().print("a");
response.getOutputStream();
```

37. Con Java 6 o superior y el JAR del conector en el classpath, ¿es obligatorio llamar a Class.forName() antes de getConnection()?

Respuesta: No. Desde JDBC 4, el controlador puede cargarse automáticamente si está correctamente instalado en el classpath.

52. ¿Qué ocurre al ejecutar esto en Tomcat 11?

Respuesta: Se agrega una cookie llamada usuario con el valor Ana Torres a la respuesta. El navegador puede almacenarla.

```
Cookie c = new Cookie("usuario", "Ana Torres");
response.addCookie(c);
```

53. Se hacen 3 peticiones GET al servlet. ¿Cuántas veces se imprime init y cuántas get?

Respuesta: init se imprime 1 vez y get se imprime 3 veces, porque init() se ejecuta al inicializar el servlet y doGet() en cada petición GET.

54. Si la clave foránea tiene ON DELETE CASCADE, ¿qué pasa al borrar a Carlos (ID 1)?

Respuesta: Se elimina a Carlos y también todos sus registros relacionados en historial_canjes.

55. ¿Qué muestra menu.jsp con este código?

Respuesta: El atributo mensaje no llega a menu.jsp, porque sendRedirect() inicia una nueva petición y se pierde el atributo de la petición anterior.

```
request.setAttribute("mensaje", "Bienvenido");
response.sendRedirect("menu.jsp");
```

56. ¿Qué campos llegan al servidor?

Respuesta: Llegan grado y juego. El campo codigo no se envía porque está deshabilitado (disabled).

```
<input name="codigo" value="A1" disabled>
<input name="grado" value="5.º" readonly>
<input name="juego" value="3" type="hidden">
```

57. Ana tiene puntos NULL. ¿Qué imprime?

Respuesta: Imprime 0 true, porque getInt() devuelve 0 cuando el valor SQL es NULL, y wasNull() confirma que el valor original era nulo.

58. Hay 10 juegos repartidos en 5 géneros, con 2 juegos por género. ¿Cuántas filas devuelve la consulta?

Respuesta: Devuelve 5 filas, una por cada género que cumple la condición HAVING COUNT(*) >= 2.

59. ¿Qué ocurre al devolver la lista de estudiantes si se cierra la conexión antes de recorrer el ResultSet?

Respuesta: Se produce un error porque la conexión se cierra antes de que el llamador recorra el ResultSet.

60. El usuario no marca la casilla. ¿Qué responde el servidor?

Respuesta: La condición es falsa y no se ejecuta el bloque, porque una casilla desmarcada no envía el parámetro acepto.

61. La tabla juegos tiene utf8mb4_unicode_ci y Fortnite y Free Fire pertenecen al género Acción. ¿Qué devuelve?

Respuesta: Devuelve 2 filas: Fortnite y Free Fire.

62. Ordenados por precio de mayor a menor, ¿qué devuelve LIMIT 3?

Respuesta: 1. Zelda: 250.
2. Mario Kart 8: 219.
3. FIFA 26: 199.90.

63. ¿Compila este servlet?

Respuesta: No compila si la excepción SQLException no se captura ni se declara en el método, porque getConnection() puede producir esa excepción.

64. ¿Qué ocurre en la última línea?

Respuesta: Se produce un error porque la segunda consulta ejecutada con el mismo Statement cierra el primer ResultSet. Por eso, r1.next() ya no puede utilizarse.

65. El servlet /login usa forward() o sendRedirect(). ¿Qué diferencia ve el usuario en la barra de direcciones?

Respuesta: Con forward(), la dirección permanece igual. Con sendRedirect(), la dirección cambia a menu.jsp.

66. ¿Qué ocurre en sendRedirect()?

Respuesta: Se produce un error porque ya se escribió y vació parte de la respuesta con flush(). No se puede redirigir correctamente después de comprometer la respuesta.

67. ¿Qué devuelve si el visitante nunca inició sesión?

Respuesta: Devuelve null, porque false indica que no se debe crear una sesión nueva si no existe.

```
HttpSession s = request.getSession(false);
```

68. ¿En qué orden salen los nombres?

Respuesta: Ana Torres (NULL), Luis Rojas (200) y Carlos Mendoza (500), porque ASC ordena de menor a mayor y MySQL coloca los valores NULL primero.

```
SELECT nombre, puntos
FROM estudiantes
ORDER BY puntos ASC;
```

69. Se marcan ambas casillas y se envía. ¿Qué contiene g?

Respuesta: request.getParameter("genero") devuelve un solo valor, no ambos. Para obtener las dos opciones se debe usar request.getParameterValues("genero").

70. Al desplegar en Tomcat 11, ¿qué ocurre con este mapeo?

Respuesta: El servlet queda asociado a la ruta /login. Si se accede a esa ruta y la aplicación está correctamente desplegada, Tomcat dirige la petición al servlet.

71. ¿Qué devuelve esta consulta para los alumnos que nunca canjearon?

Respuesta: Si la subconsulta devuelve algún NULL, la consulta exterior no devuelve filas, debido al comportamiento de NOT IN con valores nulos.

```
SELECT nombre FROM estudiantes
WHERE id_estudiante NOT IN (
    SELECT id_estudiante FROM historial_canjes
);
```

72. ¿Qué ocurre con getGeneratedKeys()?

Respuesta: Permite obtener las claves generadas automáticamente por la base de datos después del INSERT, como el ID asignado al nuevo estudiante Zoe.

73. Un compañero arma un login con este formulario. ¿Cuál es el problema principal?

Respuesta: Se envían el usuario y la contraseña mediante la URL, donde pueden quedar visibles en el historial o los registros. Para enviar credenciales se debe utilizar HTTPS y el método POST.

```
<form action="login" method="GET">
    <input name="usuario">
    <input name="clave" type="password">
</form>
```

74. MainRankingWeb registra el servlet en /ranking/*. ¿Qué responde Tomcat al acceder a http://localhost:8081/ranking/?

Respuesta: Tomcat reconoce la ruta como válida para el servlet, porque /ranking/ coincide con el patrón /ranking/*.

75. Las recompensas tienen 2 filas, con costos de 150 y 300. ¿Qué ocurre?

Respuesta: Se produce un error porque la subconsulta devuelve dos valores y el operador = espera un solo valor.

76. ¿Compila este código en Java 8 o superior?

Respuesta: No compila en Java 8, porque no se permite usar una variable declarada previamente como recurso de esa manera. Desde Java 9 sí se permite si la variable es final o efectivamente final.

```
Connection conn = ConexionBD.obtenerConexion();
try (conn) {
    // usar conn
}
```

77. Un alumno quiere reutilizar el DAO para varias tablas. ¿Qué ocurre?

Respuesta: Produce un error SQL, porque los parámetros ? sirven para valores, no para nombres de tablas. El nombre de la tabla debe formar parte de la consulta SQL.

```
ps = conn.prepareStatement("SELECT * FROM ?");
ps.setString(1, "estudiantes");
rs = ps.executeQuery();
```

78. En Linux se creó la base de datos gamestore_db en minúsculas. ¿Qué ocurre?

Respuesta: La conexión funciona si la URL utiliza exactamente gamestore_db y las demás credenciales son correctas.

79. La tabla tiene 3 estudiantes. ¿Qué imprime este código?

Respuesta: Imprime solamente el nombre del primer estudiante, porque rs.next() se ejecuta una sola vez.

```
ResultSet rs = ps.executeQuery();
if (rs.next()) {
    System.out.println(rs.getString("nombre"));
}
```

80. ¿Qué promedio devuelve? Datos: Carlos Mendoza = 500, Ana Torres = NULL, Luis Rojas = 200.

Respuesta: Devuelve 350 puntos, porque AVG() ignora los valores NULL y calcula el promedio entre 500 y 200.

```
SELECT AVG(puntos) FROM estudiantes;
```

*Se omitieron algunas preguntas y respuestas porque ya están resueltas en la evaluación en físico.

Vania P. Cruz Arias